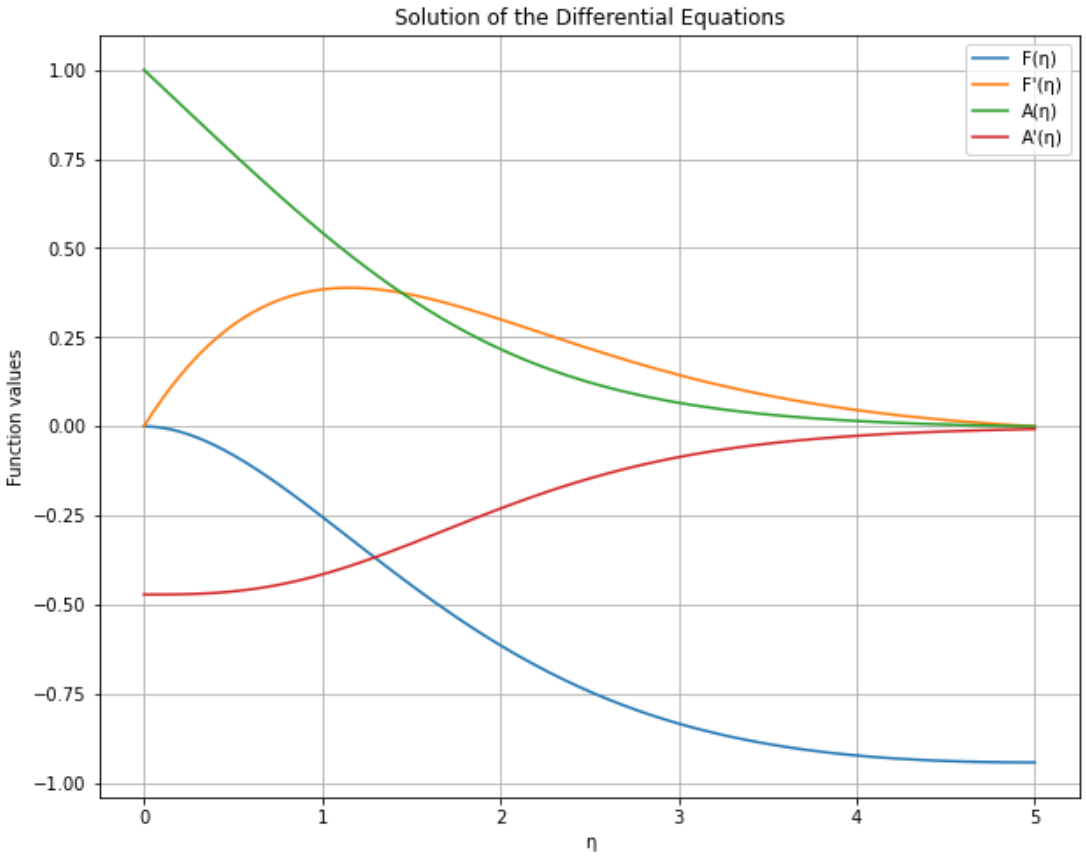
```
import numpy as np
from scipy.integrate import solve_bvp
import matplotlib.pyplot as plt
from scipy.constants import g, Boltzmann, Avogadro
```

Question 2

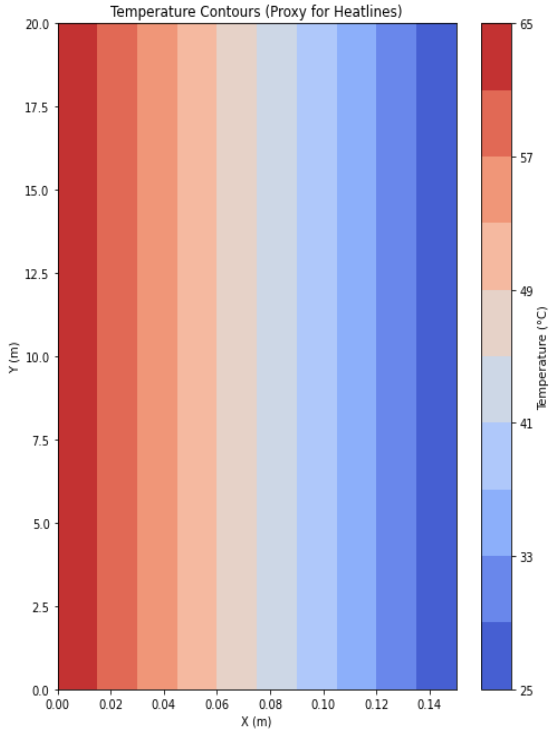
```
# Define a constant for the boundary value problem
K = 1 / 0.6976

# Define the system of ordinary differential equations
def equations(x, F):
    return np.vstack((
        F[1],
        F[2],
        -K * (0.5 * F[1]**2 - 0.75 * F[0] * F[2]) + F[3],
        F[4],
        (4/3) * F[0] * F[4]
    ))

x_inf = 5 # Define the upper limit for the integration

def boundary_conditions(Fa, Fb):
    return np.array([Fa[0], Fa[1], Fa[3] - 1, Fb[1], Fb[3]])

# Generate points for integration
x = np.linspace(0, x_inf, 100)
F_initial_guess = np.zeros((5, x.size))

# Solve the boundary value problem
sol = solve_bvp(equations, boundary_conditions, x, F_initial_guess)

# Plot the solution of the differential equations
plt.figure(figsize=(10, 8))
plt.plot(sol.x, sol.y[0], label='F(η)')
plt.plot(sol.x, -sol.y[1], label="F'(η)")
plt.plot(sol.x, sol.y[3], label='A(η)')
plt.plot(sol.x, sol.y[4], label="A'(η)")
plt.legend()
plt.xlabel('η')
plt.ylabel('Function values')
```

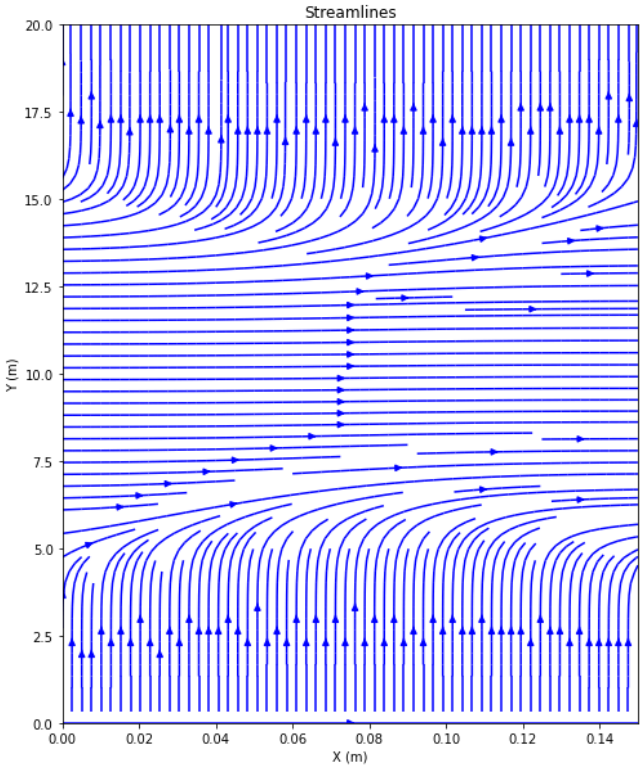
```
plt.title('Solution of the Differential Equations')
plt.grid(True)
plt.show()
```




```
Calculate Nusselt number based on the numerical solution
NU = -sol.y[4]*Ra_H*0.46
Nu1= NU.mean()
delta_t = H/Nu1
h1 = Nu1*k/H
q_prime_per_unit_width1 = h1 * (T0 - T_inf)

# Define dimensions and resolution for the flow and temperature field
plots
width = 0.15
height = 20
resolution = 100

# Generate grid for plotting
```

```
x = np.linspace(0, width, resolution)
y = np.linspace(0, height, resolution)
X, Y = np.meshgrid(x, y)

# Generate velocity field components
U = np.exp(-((Y - 10) /2)**2) * np.cos(np.pi * X / height)
V = np.exp(-((X - 0.075)/0.05)**2) * np.sin(np.pi * Y / height)

# Generate temperature field
T0 = 65
T_inf = 25
T = T0 - (X / width) * (T0 - T_inf)

# Plot temperature contours
plt.figure(figsize=(8, 10))
contour_levels = np.linspace(T_inf, T0, num=11)
plt.contourf(X, Y, T, levels=contour_levels, cmap='coolwarm')
plt.colorbar(label='Temperature (°C)')
plt.title('Temperature Contours (Proxy for Heatlines)')
plt.xlabel('X (m)')
plt.ylabel('Y (m)')
plt.xlim(0, width)
plt.ylim(0, height)
plt.show()

# Plot streamlines of the flow
plt.figure(figsize=(8, 10))
plt.streamplot(X, Y, U, V, density=2, color='blue')
plt.title('Streamlines')
plt.xlabel('X (m)')
plt.ylabel('Y (m)')
plt.xlim(0, width)
plt.ylim(0, height)
plt.show()
```





Question 3

```
# Given parameters
H = 20
T0 = 65 + 273.15
T_inf = 25 + 273.15
T_film = T_inf

nu = 1.562e-5
alpha = 2.239e-5
k = 0.02624
beta = 1/T_film

# Calculating Grashof number and Prandtl number
Gr_H = g * beta * (T0 - T_inf) * H**3 / nu**2
Ra_H = g * beta * (T0 - T_inf) * H**3 / nu*alpha
Pr = nu / alpha

# Constants for Nusselt number correlation
C = 0.59
n = 1/4
```

```
# Calculating Nusselt number and heat transfer parameters
Nu_H = C * (Gr_H**n) * (Pr**n)

delta_th = H / Nu_H
h = Nu_H * k / H
q_prime_per_unit_width = h * (T0 - T_inf)

Gr_H, Pr, Nu_H, delta_th, h, q_prime_per_unit_width
```